LAPORAN PRAKTIKUM PBO

PRAKTIKUM 6 Abstraction: Abstract Class, Abstract Method, Interface, Interface Inheritance, Default Method, etc.

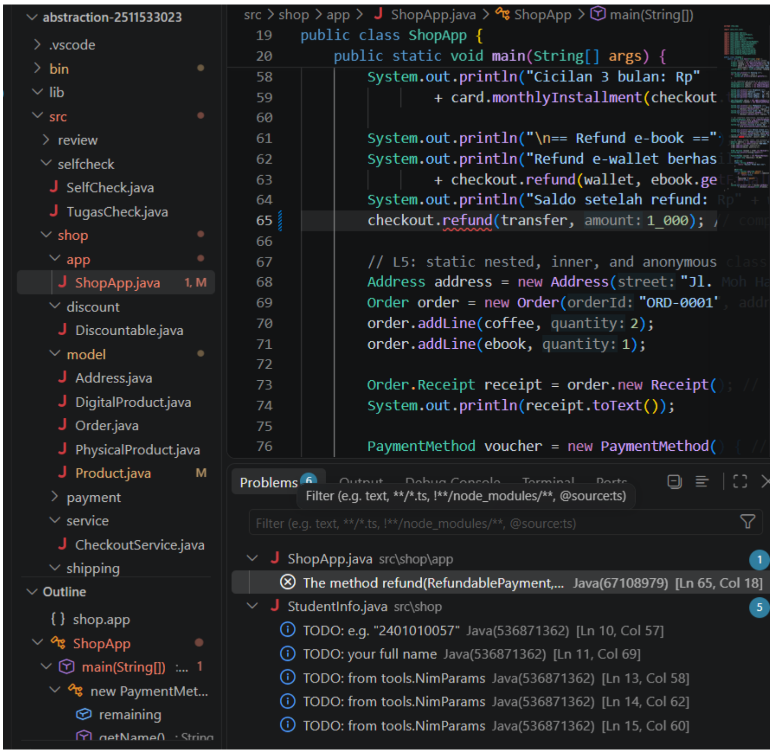
Disusun oleh :
M. Fajar Fadhilul Zikri
NIM: 2511533023
Dosen Pengampu: **AFDHAL DINILHAK, M.Kom**
Asisten Praktikum: **Muhammad Dawi Syauqi**

FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
DEPARTEMEN INFORMATIKA
TAHUN 2026

A. Tahapan Pelaksanaan Tugas PBO

Compiler L1,L3, dan L4

L3:

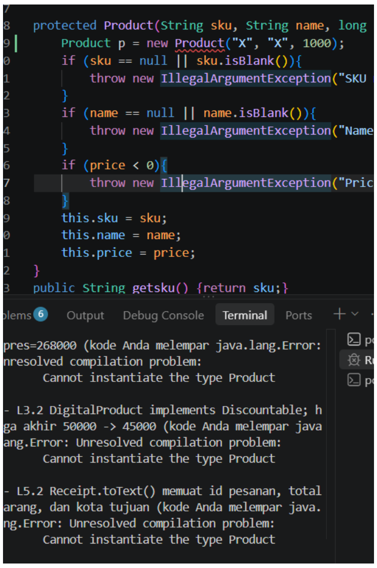

abstraction-2511533023
.vscode
bin
lib
src
review
selfcheck
SelfCheck.java
TugasCheck.java
shop
app
ShopApp.java 1, M
discount
Discountable.java
model
Address.java
DigitalProduct.java
Order.java
PhysicalProduct.java
Product.java M
payment
service
CheckoutService.java
shipping
Outline
{ } shop.app
ShopApp
main(String[]) :... 1
new PaymentMet...
remaining
getName() : String
src > shop > app > ShopApp.java > ShopApp > main(String[])
Problems 6
Output
Debug Console
Terminal
Ports

```
public class ShopApp {
    public static void main(String[] args) {
        System.out.println("Cicilan 3 bulan: Rp"
                + card.monthlyInstallment(checkout

        System.out.println("\n== Refund e-book =="
        System.out.println("Refund e-wallet berhas
                + checkout.refund(wallet, ebook.ge
        System.out.println("Saldo setelah refund:
        checkout.refund(transfer, amount: 1_000);

        // L5: static nested, inner, and anonymous
        Address address = new Address(street: "Jl.
        Order order = new Order(orderId: "ORD-0001"
        order.addLine(coffee, quantity: 2);
        order.addLine(ebook, quantity: 1);

        Order.Receipt receipt = order.new Receipt(
        System.out.println(receipt.toText());

        PaymentMethod voucher = new PaymentMethod(
```

```
Filter (e.g. text, **/*.ts, !**/node_modules/**, @source:ts)

ShopApp.java src\shop\app                                                1
    The method refund(RefundablePayment,...  Java(67108979)  [Ln 65, Col 18]
StudentInfo.java src\shop                                                5
    TODO: e.g. "2401010057"  Java(536871362)  [Ln 10, Col 57]
    TODO: your full name  Java(536871362)  [Ln 11, Col 69]
    TODO: from tools.NimParams  Java(536871362)  [Ln 13, Col 58]
    TODO: from tools.NimParams  Java(536871362)  [Ln 14, Col 62]
    TODO: from tools.NimParams  Java(536871362)  [Ln 15, Col 60]
```

Method refund pada CheckoutService mengharapkan argumen bertipe antarmuka RefundablePayment, tetapi diberikan objek transfer bank biasa yang tidak mengimplementasikan interface tersebut. Hal ini memicu kesalahan kompiler karena tipe data parameter tidak sesuai dengan kontrak method.

L1:


Problems 6
Output
Debug Console
Terminal
Ports

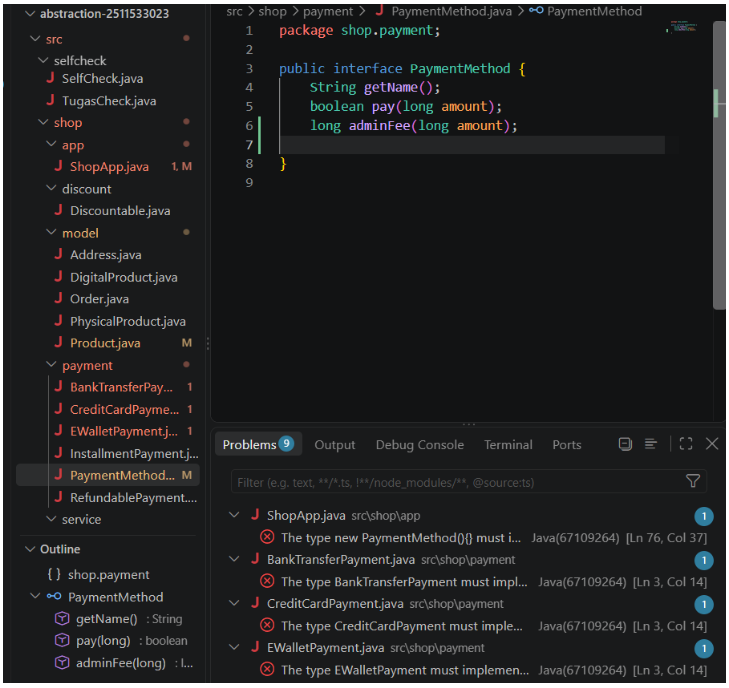
```
protected Product(String sku, String name, long
    Product p = new Product("X", "X", 1000);
    if (sku == null || sku.isBlank()){
        throw new IllegalArgumentException("SKU
    }
    if (name == null || name.isBlank()){
        throw new IllegalArgumentException("Name
    }
    if (price < 0){
        throw new IllegalArgumentException("Pric
    }
    this.sku = sku;
    this.name = name;
    this.price = price;
}
public String getsku() {return sku;}
```

```
pres=268000 (kode Anda melempar java.lang.Error:
nresolved compilation problem:
     Cannot instantiate the type Product

- L3.2 DigitalProduct implements Discountable; h
ga akhir 50000 -> 45000 (kode Anda melempar java
ang.Error: Unresolved compilation problem:
     Cannot instantiate the type Product

- L5.2 Receipt.toText() memuat id pesanan, total
arang, dan kota tujuan (kode Anda melempar java.
ng.Error: Unresolved compilation problem:
     Cannot instantiate the type Product
```

Product bersifat *abstract*, sehingga Java melarang instansiasi objek secara langsung dari *abstract class*.

L4:

T1.a dan T1.b:


abstraction-2511533023
src
selfcheck
SelfCheck.java
TugasCheck.java
shop
app
ShopApp.java 1, M
discount
Discountable.java
model
Address.java
DigitalProduct.java
Order.java
PhysicalProduct.java
Product.java M
payment
BankTransferPay... 1
CreditCardPayme... 1
EWalletPayment.j... 1
InstallmentPayment.j...
PaymentMethod... M
RefundablePayment....
service
Outline
{ } shop.payment
PaymentMethod
getName() : String
pay(long) : boolean
adminFee(long) : l...
src > shop > payment > PaymentMethod.java > PaymentMethod
Problems 9
Output
Debug Console
Terminal
Ports
Filter (e.g. text, **/*.ts, !**/node_modules/**, @source:ts)

```java
package shop.payment;

public interface PaymentMethod {
    String getName();
    boolean pay(long amount);
    long adminFee(long amount);

}

```

```
ShopApp.java src\shop\app
    The type new PaymentMethod(){} must i...   Java(67109264)  [Ln 76, Col 37]
BankTransferPayment.java src\shop\payment
    The type BankTransferPayment must impl...   Java(67109264)  [Ln 3, Col 14]
CreditCardPayment.java src\shop\payment
    The type CreditCardPayment must imple...   Java(67109264)  [Ln 3, Col 14]
EWalletPayment.java src\shop\payment
    The type EWalletPayment must implemen...   Java(67109264)  [Ln 3, Col 14]
```

Error terjadi karena konstanta CARD_ADMIN_PERCENT belum diimpor atau dideklarasikan di dalam scope interface PaymentMethod dengan batasan NIM yang sesuai. Method adminFee(...) memerlukan rujukan konstanta yang tepat dari kontrak interface agar perhitungan biaya admin 1% kartu kredit dapat berjalan.

T1.c:

Ini untuk credit card

```
@Override
    public long adminFee(long amount) {
        return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
    }
```

ini untuk bank transfer:

```
@Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
```

T1.d:

Kita menambahkan method grandTotal dan mengubah atau memperbarui checkout sehingga dapat menghitung total keseluruhan (*grand total* yang sudah mencakup harga barang, ongkos kirim, dan biaya admin metode pembayaran) lalu mengeksekusi pembayaran tersebut.

T1.e:

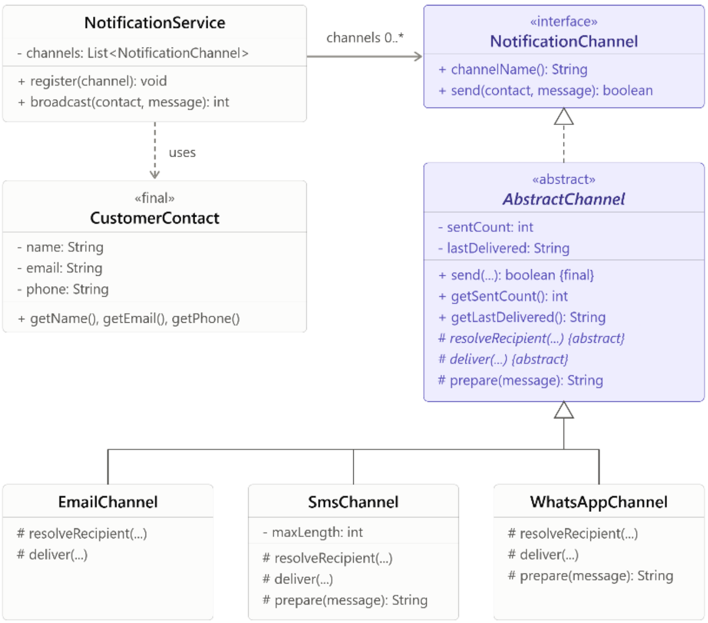
Penambahan default method policy() bertujuan untuk menciptakan konflik diamond ketika sebuah class (seperti CreditCardPayment) mengimplementasikan dua interface sekaligus. Hal ini memaksa pemrogram untuk melakukan *override* manual guna menggabungkan kedua kebijakan menggunakan sintaks .super.policy().

Pertanyaan :

1) Mengapa EWalletPayment tidak perlu meng-override policy()?
   Karena EWalletPayment hanya mengimplementasikan RefundablePayment (tidak mengimplementasikan InstallmentPayment), sehingga tidak ada bentrok atau ambiguitas asal method policy().
2) Mengapa Java tidak memilih salah satu default secara otomatis?
   Untuk menghindari ambiguitas (*ambiguity*) dan perilaku program yang tidak terduga (*unexpected behavior*). Java mewajibkan developer untuk membuat keputusan secara eksplisit agar desain kode tetap aman dan konsisten.

T2: pada bagian ini kita akan membuat / merancang system notifikasi pesanan Langkah awal buat package baru di dalam shop yakni notification (shop.notification) yang didalam nya terdapat 7 class yakni abstractChannel, customercontact,EmailChannel,NotificationChannel,SmsChannel,WhatsappChannel,dan NotificationService. Dan untuk demo kita buat di package app untuk menguji apakah sudah sesuai atau belum.

Berikut gambar diagram UMLnya:


NotificationService
- channels: List<NotificationChannel>
+ register(channel): void
+ broadcast(contact, message): int
channels 0..*
«interface»
NotificationChannel
+ channelName(): String
+ send(contact, message): boolean
uses
«final»
CustomerContact
- name: String
- email: String
- phone: String
+ getName(), getEmail(), getPhone()
«abstract»
AbstractChannel
- sentCount: int
- lastDelivered: String
+ send(...): boolean {final}
+ getSentCount(): int
+ getLastDelivered(): String
# resolveRecipient(...) {abstract}
# deliver(...) {abstract}
# prepare(message): String
EmailChannel
# resolveRecipient(...)
# deliver(...)
SmsChannel
- maxLength: int
# resolveRecipient(...)
# deliver(...)
# prepare(message): String
WhatsAppChannel
# resolveRecipient(...)
# deliver(...)
# prepare(message): String

Pertanyaan T2:

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

Keberadaan abstract class dan interface secara bersamaan diperlukan karena keduanya memiliki peran yang saling melengkapi; abstract class digunakan untuk mendefinisikan kerangka dasar, constructor, dan field bersama yang menunjukkan hubungan hierarki jenis (is-a), sedangkan interface mendefinisikan kontrak kemampuan atau perilaku (can-do) yang dapat diimplementasikan oleh berbagai class lintas hierarki.

2. Apa akibatnya jika send(...) tidak dibuat final?

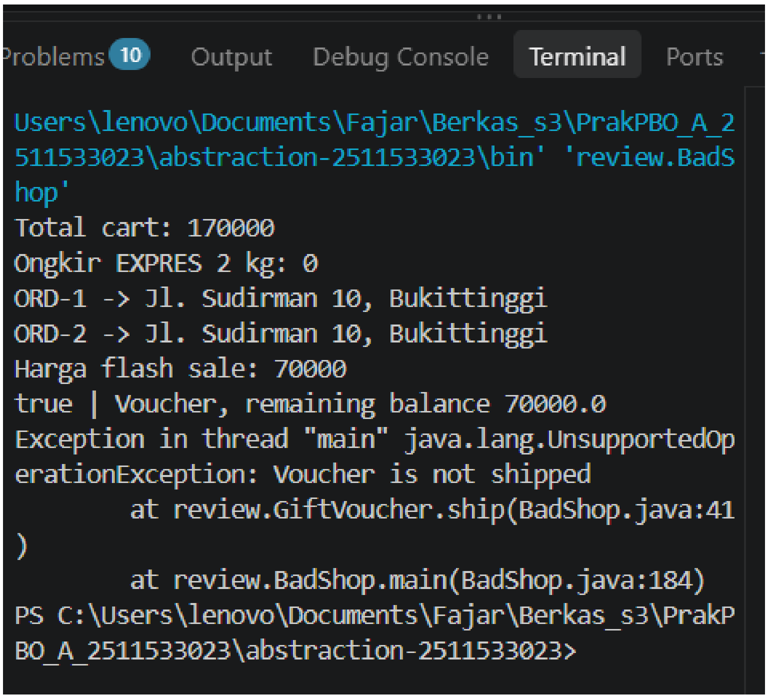
subclass memiliki celah untuk mengubah atau menimpa (override) alur logika utama (template method) yang telah dirancang—seperti validasi pesan kosong, penyaringan alamat, dan pencatatan jumlah pengiriman—sehingga dapat merusak konsistensi aturan pengiriman pesan yang seharusnya bersifat terpusat.

3. Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

Untuk menerapkan prinsip pemrograman terhadap antarmuka (program to an interface), sehingga layanan dapat fleksibel menerima berbagai bentuk kanal baru tanpa harus bergantung pada hierarki class tertentu, termasuk mendukung penggunaan anonymous class seperti kanal log konsol yang tidak mewarisi abstract class tersebut.

T3Code Review BadShop:

Sebelum refactor:



```
Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_2
511533023\abstraction-2511533023\bin' 'review.BadS
hop'
Total cart: 170000
Ongkir EXPRES 2 kg: 0
ORD-1 -> Jl. Sudirman 10, Bukittinggi
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | Voucher, remaining balance 70000.0
Exception in thread "main" java.lang.UnsupportedOp
erationException: Voucher is not shipped
        at review.GiftVoucher.ship(BadShop.java:41
)
        at review.BadShop.main(BadShop.java:184)
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakP
BO_A_2511533023\abstraction-2511533023>
```

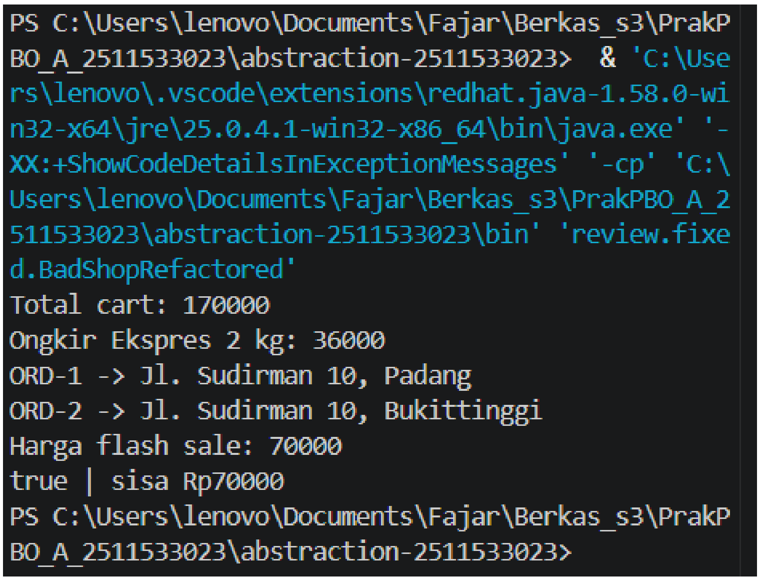
Tabel:

No	Lokasi/class	masalah	Dampak nyata	Perbaikan yang diusulkan
1	StoreOperations & GiftVoucher	interface Fat (Pelanggaran ISP):	Program *crash* saat	Pecah interface

		GiftVoucher dipaksa mengimplementasikan method yang tidak relevan (ship, refund).	method tersebut dipanggil karena melempar	menjadi lebih spesifik sesuai kemampuan objek (*Interface Segregation*).
2	Courier.cost(...)	Anti-Polimorfisme (String-based Switch): Penentuan tarif kurir menggunakan percabangan string ("EXPRES").	Rawan *typo* (seperti string EXPRES pada *main*), menyebabkan ongkos kirim bernilai 0.	Gunakan polimorfisme dengan *abstract class* atau *interface* kurir dan subclass konkret.
3	Address	Pelanggaran Immutabilitas: Address memiliki method *setter* (setCity, setStreet).	Alamat pada pesanan pertama (ORD-1) ikut berubah tanpa sengaja saat objek alamat dimodifikasi di pesanan kedua.	Jadikan Address sebagai *final class* dengan field *private final* tanpa *setter*.
4	Discount.apply(...)	Anti-Polimorfisme (instanceof): Logika diskon	Melanggar prinsip OOP murni; kode menjadi kaku dan	Ubah menjadi method

		menggunakan pemeriksaan instanceof di dalam *default method*.	sulit diperluas saat ada jenis diskon baru.	abstrak atau terapkan polimorfisme langsung di tiap subclass diskon.
5	Cart.Item	Inner Class Tanpa Aturan Akses yang Tepat: Item dideklarasikan sebagai inner class biasa di dalam Cart.	Keterikatan struktur data menjadi terlalu ketat dan kurang fleksibel untuk diuji secara independen.	Jadikan *static nested class* atau class terpisah di package *model*.

Sesudah refactor:



```
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakP
BO_A_2511533023\abstraction-2511533023>  & 'C:\Use
rs\lenovo\.vscode\extensions\redhat.java-1.58.0-wi
n32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-
XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\
Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_2
511533023\abstraction-2511533023\bin' 'review.fixe
d.BadShopRefactored'
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakP
BO_A_2511533023\abstraction-2511533023>
```

Bagian yang saya refactor ada 3:

1. Address (Perbaikan Mutabilitas & Shared-Reference Bug)

menghilangkan method setter (setCity dan setStreet) yang sebelumnya ada pada kode sumber. Class ini sekarang dibuat bersifat immutable dengan field berjenis final guna mencegah terjadinya shared-reference mutation, di mana perubahan alamat pada objek pesanan kedua (ORD-2) sebelumnya dapat ikut merubah data pada pesanan pertama (ORD-1).

2.Courier

menghapus pendekatan string-based switch yang rawan terhadap kesalahan pengetikan (typo) di dalam class Courier. Logika penentuan tarif tersebut kini digantikan dengan polimorfisme murni menggunakan class abstrak dan pembuatan subclass konkret seperti ExpressCourier untuk mengeksekusi perhitungan biaya kirim secara aman.

3.Discount

Perbaikan nya terletak pada interface discount yang sebelumnya menggunakan percabangan pemeriksaan instanceof secara manual di dalam *default method*. Anti-pattern tersebut telah dihapus dan digantikan dengan penerapan method abstrak apply() yang dioverride secara langsung di masing-masing subclass, seperti FlashSaleDiscount dan MemberDiscount, agar struktur kode menjadi lebih fleksibel.

T4 Refleksi

Tabel 1 – Abstract class vs interface

Pertanyaan desain	Abstract class / interface?	Alasan	Contoh dari proyek
Subclass berbagi field dan konstruktor	Abstract class	Mendukung state Bersama field dan konstruktor untuk inisialisai awal.	Product yang diextends oleh physical dan digital product.
Satu class perlu beberapa	Interface	mendukung multiple inheritance of	CreditCardPayment implement

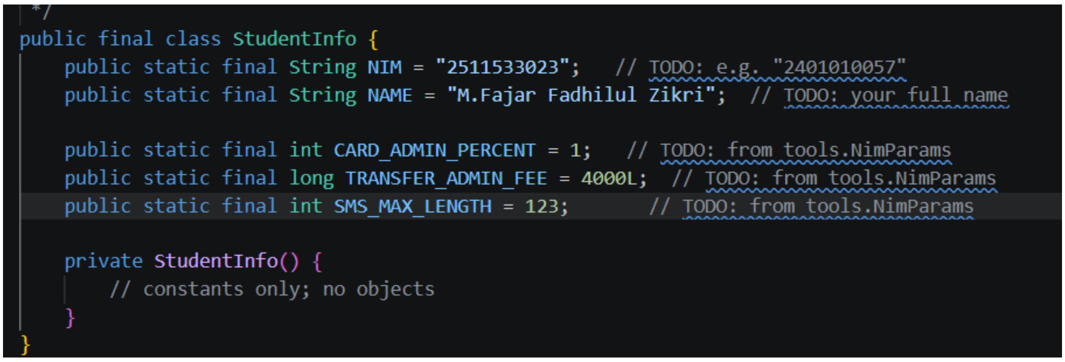
kemampuan sekaligus		type melalui interface (mengatasi batasan single inheritance).	RefundablePayment dan InstallmentPayment
Rumus bersama yang tidak boleh diubah subclass	Abstract class	Memungkinkan penerapan metode konkret atau dikunci dengan kata kunci final agar alur konsisten.	AbstractChannel.send atau ShippingService.calculateCost
Kontrak yang perlu ditambah method baru di kemudian hari	Interface dengan menggunakan default	Memungkinkan penambahan method baru tanpa merusak implementasi class yang sudah ada.	RefundablePayment.policy()

Tabel 2 – Jenis nested class

Kondisi	Static nested, inner,/ anonymous?	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	Static nested class	Tidak memerlukan referensi ke instance outer class sehingga	Helper / Utility internal di dalam modul

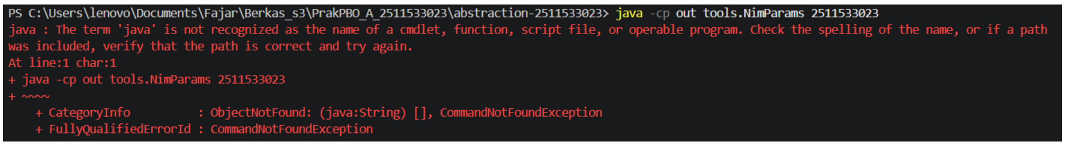
		lebih efisien memori.	
Objek harus membaca field objek luarnya	Inner class (non-static)	Memiliki referensi implisit ke outer class sehingga dapat mengakses instance field secara langsung	Cart.Item di dalam class Cart
Objek harus membaca field objek luarnya	Anonymous class	Memungkinkan implementasi interface atau abstract class secara langsung di tempat tanpa membuat class baru.	Kanal log konsol di TugasApp.java (new NotificationChannel() { ... })

Identitas: saya hitung manual menggunakan rumus pada file:



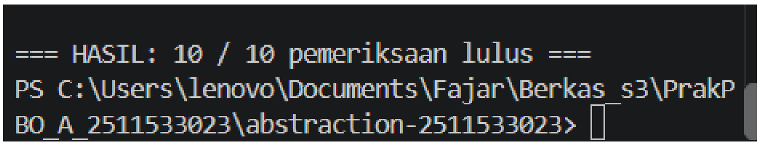
```java
 */
public final class StudentInfo {
    public static final String NIM = "2511533023";   // TODO: e.g. "2401010057"
    public static final String NAME = "M.Fajar Fadhilul Zikri";  // TODO: your full name

    public static final int CARD_ADMIN_PERCENT = 1;   // TODO: from tools.NimParams
    public static final long TRANSFER_ADMIN_FEE = 4000L;  // TODO: from tools.NimParams
    public static final int SMS_MAX_LENGTH = 123;       // TODO: from tools.NimParams

    private StudentInfo() {
        // constants only; no objects
    }
}
```

Untuk tools.NimParams saya tidak bisa jalan ini buktinya:



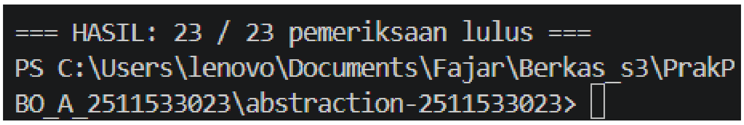
```
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_2511533023\abstraction-2511533023> java -cp out tools.NimParams 2511533023
java : The term 'java' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path
was included, verify that the path is correct and try again.
At line:1 char:1
+ java -cp out tools.NimParams 2511533023
+ ~~~~
    + CategoryInfo          : ObjectNotFound: (java:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

Screenshot TugasCheck:



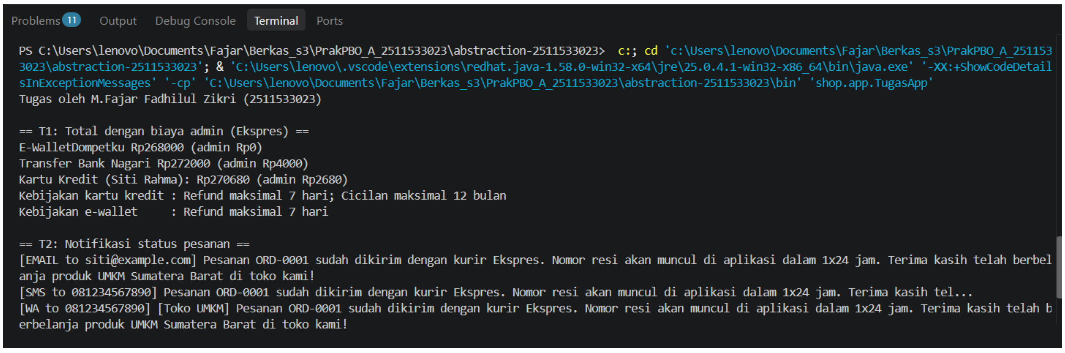
```
=== HASIL: 10 / 10 pemeriksaan lulus ===
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakP
BO_A_2511533023\abstraction-2511533023>
```

Screenshot SelfCheck:

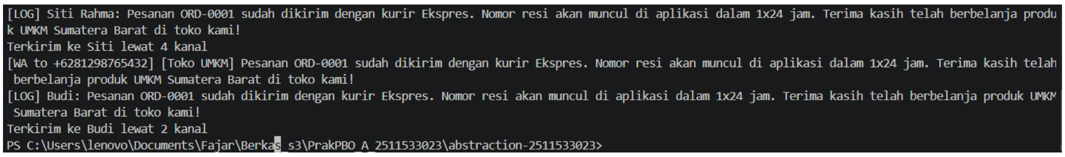


```
=== HASIL: 23 / 23 pemeriksaan lulus ===
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakP
BO_A_2511533023\abstraction-2511533023>
```

Screenshot/output TugasApp:



```
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_2511533023\abstraction-2511533023>  c:; cd 'c:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_251153
3023\abstraction-2511533023'; & 'C:\Users\lenovo\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetail
sInExceptionMessages' '-cp' 'C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_2511533023\abstraction-2511533023\bin' 'shop.app.TugasApp'
Tugas oleh M.Fajar Fadhilul Zikri (2511533023)

== T1: Total dengan biaya admin (Ekspres) ==
E-WalletDompetku Rp268000 (admin Rp0)
Transfer Bank Nagari Rp272000 (admin Rp4000)
Kartu Kredit (Siti Rahma): Rp270680 (admin Rp2680)
Kebijakan kartu kredit : Refund maksimal 7 hari; Cicilan maksimal 12 bulan
Kebijakan e-wallet     : Refund maksimal 7 hari

== T2: Notifikasi status pesanan ==
[EMAIL to siti@example.com] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbel
anja produk UMKM Sumatera Barat di toko kami!
[SMS to 081234567890] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih tel...
[WA to 081234567890] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah b
erbelanja produk UMKM Sumatera Barat di toko kami!
```



```
[LOG] Siti Rahma: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produ
k UMKM Sumatera Barat di toko kami!
Terkirim ke Siti lewat 4 kanal
[WA to +6281298765432] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah
 berbelanja produk UMKM Sumatera Barat di toko kami!
[LOG] Budi: Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM
 Sumatera Barat di toko kami!
Terkirim ke Budi lewat 2 kanal
PS C:\Users\lenovo\Documents\Fajar\Berkas_s3\PrakPBO_A_2511533023\abstraction-2511533023>
```